Kurs Django — Lekcja 6

Relacje między modelami

Cel lekcji

Student rozumie i potrafi zastosować ForeignKey, ManyToManyField i OneToOneField, wykonuje zapytania po relacjach i wyświetla powiązane dane w szablonach.

Blok 1: Teoria (15 min)

Trzy typy relacji

- ForeignKey (jeden-do-wielu) — np. jeden autor może mieć wiele postów. Pole umieszczamy w modelu „po stronie wielu” (Post).
- ManyToManyField (wiele-do-wielu) — np. post może mieć wiele tagów, a tag może być przypisany do wielu postów. Django tworzy automatycznie tabelę pośredniczącą.
- OneToOneField (jeden-do-jednego) — np. rozszerzenie profilu użytkownika o dodatkowe dane (jeden użytkownik = jeden profil).

related_name

Pozwala nazwać „odwrotną” relację — dzięki temu z obiektu Kategoria możemy łatwo dostać się do wszystkich jej postów (kategoria.posty.all()) zamiast domyślnego, mniej czytelnego kategoria.post_set.all().

Blok 2: Ćwiczenia w terminalu

Krok 1 — model Kategoria i relacja ForeignKey

W blog/models.py:

```
class Kategoria(models.Model):
    nazwa = models.CharField(max_length=100)

    def __str__(self):
        return self.nazwa

    class Meta:
        verbose_name_plural = "Kategorie"


class Post(models.Model):
    tytul = models.CharField(max_length=200)
    tresc = models.TextField()
    data_publikacji = models.DateTimeField(auto_now_add=True)
    opublikowany = models.BooleanField(default=True)
    kategoria = models.ForeignKey(
        Kategoria,
```

```
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='posty'
    )
```

Omówić on_delete — co ma się stać z postami, gdy kategoria zostanie usunięta:

- CASCADE — usuń też powiązane posty
- SET_NULL — ustaw pole na NULL (wymaga null=True)
- PROTECT — zablokuj usunięcie kategorii, dopóki są przypisane posty

Krok 2 — migracja

```
python manage.py makemigrations
python manage.py migrate
```

Krok 3 — rejestracja w adminie

W blog/admin.py:

```
from .models import Post, Kategoria

admin.site.register(Kategoria)
```

Krok 4 — relacja ManyToMany (tagi)

```
class Tag(models.Model):
    nazwa = models.CharField(max_length=50)

    def __str__(self):
        return self.nazwa


class Post(models.Model):
    ...
    tagi = models.ManyToManyField(Tag, blank=True, related_name='posty')
python manage.py makemigrations
python manage.py migrate
```

Zarejestrować Tag w adminie.

Krok 5 — zapytania po relacjach w shellu

```
python manage.py shell
from blog.models import Post, Kategoria, Tag

# utworzenie kategorii i przypisanie
k = Kategoria.objects.create(nazwa="Technologia")
post = Post.objects.first()
post.kategoria = k
post.save()

# dostęp "w przód"
post.kategoria.nazwa

# dostęp "wstecz" dzięki related_name
```

```
k.posty.all()

# many-to-many
t1 = Tag.objects.create(nazwa="python")
t2 = Tag.objects.create(nazwa="django")
post.tagi.add(t1, t2)
post.tagi.all()

# wstecz dla many-to-many
t1.posty.all()
```

Krok 6 — wyświetlanie relacji w szablonie

W widoku (blog/views.py):

```
def strona_glowna(request):
    posty = Post.objects.filter(opublikowany=True)
    return render(request, 'blog/strona_glowna.html', {'posty': posty})
```

W szablonie:

```
{% for post in posty %}
    <article>
        <h2>{{ post.tytul }}</h2>
        {% if post.kategoria %}
            <p>Kategoria: {{ post.kategoria.nazwa }}</p>
        {% endif %}
        <p>Tagi:
            {% for tag in post.tagi.all %}
                <span>{{ tag.nazwa }}</span>
            {% endfor %}
        </p>
    </article>
{% endfor %}
```

Blok 3: Zadanie dla studentów

- Dodać model Komentarz z ForeignKey do Post (on_delete=models.CASCADE) i polami autor, tresc, data_dodania
- W shellu dodać kilka komentarzy do jednego posta i wypisać je przez post.komentarze.all() (ustawić odpowiedni related_name)
- W szablonie strona_glowna.html wyświetlić liczbę komentarzy przy każdym poście ({{ post.komentarze.count }})

Checklist nawyków

☐ Każda relacja i zmiana w modelach → makemigrations + migrate

☐ ForeignKey zawsze ma jawnie ustawione on_delete

☐ Nowy model powiązany → pamiętać o rejestracji w admin.py